

Fuel Pressure Sensor and Alarm Module Low-Level Design (LLD)

Version: 1.0

Date: 17-03-2026

Prepared by: Karthika V T

1. Purpose

This document provides a detailed blueprint for implementing each component of the Fuel Pressure and Alarm system. While the HLD outlines the system architecture and major modules, the LLD dives into the internal logic, data structures, algorithms and interface specifications required to build those modules. It translates the abstract design into concrete actionable development plans.

2. Functional Decomposition

2.1 Sensor Interface Module

It acquires and validates fuel pressure data from the sensor. Several APIs are present in the sensor driver that initialize sensor hardware, read raw analog voltage, convert it to the corresponding pressure value. A struct variable is used to store the sensor data, the pressure initialisation function is used to initialize the ADC and pressure sensor. In the Sensor read function a buffer of predefined length is taken and the sensor data is shared towards the buffer and the sensor value is returned to the caller function.

2.2 Validation Module

The module consists of the struct variable for intaking the pressure values and an Queue handler variable and an event handler variable, both declared as extern since it is to be shared with the alarm module. Both maximum and minimum limit of pressure is predefined. In the pressure validation function, the pressure is received to the function using the queue receive function. The pressure value is compared with minimum and maximum values. If the condition is met an Event signal is generated and given to the alarm parameter. After a short delay the event signal bit will be cleared.

2.3 Alarm Module

The module is triggered using an event signal. The alarmTask function sets the alarm registers for a predefined time and resets after the duration.

3. Detailed Function Descriptions

3.1 Sensor Module

3.1.1 Function prototypes

```
bool PressureSensor_Init(void)
```

```
uint32_t PressureSensor_Read(void)
```

```
bool PressureProducerTask( * pressureParameters)
```

3.1.2 Pseudocode

Define structure PressureData with pressure and timestamp variables.

Function PressureSensor_Init reads the chip_id from the sensor. If the return value is not expected, an error flag will be set. Disable the sensor before configuring, and configure the pressure sensor with corresponding registers for sampling rate, mode, time, etc. Return Success on exit of function.

Function PressureSensor_Read reads the raw pressure from the sensor and converts the raw value to actual pressure value. The function returns the actual pressure.

The handler function PressureProducerTask declares a data type of PressureData. The function calls the initialization function. If initialization fails, an error flag is set. In a forever while loop, read pressure data to the variable data.pressure by calling PressureSensor_Read function. The timestamp parameter will get value from the systick timer where the time corresponds to the time at which pressure value is taken. Share the data to the message queue using the queue variable and a fixed time delay is given(eg. 1s)

3.2 Validation Module

3.2.1 Function prototypes

```
bool PressureValidationTask(pressureParameter)
```

3.2.2 Pseudocode

The threshold value of pressure(MIN_PRESSURE and MAX_PRESSURE) is defined. Inside the PressureValidationTask function, declare a receiveData of PressureData. In a forever while loop, receive the data from the message queue which is shared from the sensor module part. If data is received successfully check the value with MIN_PRESSURE and MAX_PRESSURE.

If pressure < MIN_PRESSURE and pressure > MAX_PRESSURE, set the alarm event bit else clear the alarm event bit.

3.3 Alarm Module

3.3.1 Function prototypes

```
bool AlarmTask(pressureParameters)
```

```
bool AlarmHandler(void)
```

3.3.2 Pseudocode

Define the PRESSURE_ALARM_BIT

Initialize the event for alarm.

In the AlarmTask, Inside a forever loop, whenever an event signal is generated, it calls the handler function for alarm triggering.

In the AlarmHandler function, the PRESSURE_ALARM_BIT is reset and the registers corresponding to the alarm buzzer are set for a particular time and it is reset.

4. Data Structures

Variable	Type	Description
bool_val	enum	provide state for success
sensor_status	enum	return success or error from sensor
pressureData_t	structure	contains sensor parameters pressure and time
pressureQueue	QueueHandle_t	Queue handler to send pressure
alarmEvent	Event object	generated to trigger alarm
SENSOR_ADDR	Constant	sensor address
QUEUE_LENGTH	Constant	number of elements on queue
MIN_PRESSURE	Constant	Min pressure value
MAX_PRESSURE	Constant	Max pressure value
ALARM_BIT	constant	Register value for alarm

5. Error Handling

5.1 Error Handling Mechanism

Validate sensor presence using the address returned from the communication protocol.

Sensor read errors - Return error code on skipping sending data or invalid data.

Sensor data is packed into a fixed structure for consistent data format, no partial or corrupted transfers between tasks.

Queue operations are used for data transfer - the return values of queue api can be checked for failures.

Validation error handling - If the pressure value falls below or above the threshold values it is treated as an error condition and an event signal is generated to indicate fault.

On receiving an error event, alarm actions are triggered and when condition returns to normal, the alarm can be cleared.

5.2 Boundary Checks

Pressure value is checked with the boundary pressure values which is pre defined by the user. If it is out of pressure range, an event signal is sent to the alarm module. The alarm is triggered for a particular time period after that the registers will be cleared.

6. Timing and Performance

6.1 Timing Requirements

Sensor sampling rate - Pressure is sampled every 1 second.

Alarm response time - as soon as the event bit is set, interrupt will be generated and corresponding interrupt handler is called.

Alarm duration - Alarm can be triggered for a particular time period that can be set using delays.

6.2 Memory usage constraints

Queue memory can be limited to particular size depending on user choice and each item will be the size of the pressure data structure. The data from the sensor is temporarily stored in a buffer and then the buffer value is copied to the queue. Static variables help reduce the scope of it to the particular function stack.

7. Test Points

PressureSensor_Init() -

If the return status of the sensor. If chip id is invalid or communication fault occurs, it returns failure.

PressureSensor_Read() -

If invalid data is sent or data is corrupted, the function returns an error flag.

PressureValidationTask(pressureParameter) -

It contains the core decision making logic, it checks whether the pressure is in a predefined range or not. An event signal is set, if it is out of range.

AlarmTask -

It sends the status flag, if the alarm is uninitialized the error flag is raised and if the alarm gets executed, the status flag is set based on the alarm state.

8. Traceability

This document has been prepared based on the SRS to ensure that all the design elements, data flow, control logic and fault management mechanisms trace back to the defined requirements.